G10旅行记忆模块python原型文档V2

vlm-travel-recorder

车载旅行记忆模块：基于窄协议设计的Agent架构，处理APP端预处理的图像，调用model server生成旅行记忆，并存储到本地SQLite数据库。

特性

- **窄协议设计**: image_list + task_name + related_params
- **Agent架构**: 负责prompt管理和业务逻辑链路
- **本地内存管理**: 使用SQLite存储多维度旅行记忆
- **模块化设计**: Agent、Model、Memory三个核心模块
- **Flask框架**: 轻量级Web服务，接近C语言的简洁性

快速开始

1. 配置 `.env` （VLLM URL、DB 路径等）
2. `docker-compose up --build`
3. POST 请求到 `/api/v1/vlm/process` 进行测试

目录结构

代码块

```
vlm-travel-recorder/
├─ README.md
├─ pyproject.toml
├─ docker/
│  ├─ Dockerfile.service
│  └─ Dockerfile.mock
├─ deploy/
│  ├─ docker-compose.yml
│  └─ k8s/
│     ├─ deployment.yaml
│     ├─ service.yaml
│     └─ hpa.yaml
├─ src/
│  ├─ app/
│  │  ├─ __init__.py
```

```
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── endpoints.py
│   │   │   │   └── schemas.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── logger.py
│   │   │   └── metrics.py
│   │   ├── services/
│   │   │   ├── vlm_client.py
│   │   │   ├── preprocessor.py
│   │   │   ├── memory.py
│   │   │   └── interior_analyzer.py
│   │   ├── models/
│   │   └── utils/
│   │       ├── image_utils.py
│   │       └── retry.py
│   └── tests/
│       ├── unit/
│       └── integration/
├── infra/
│   ├── prometheus/
│   └── grafana/
├── ci/
│   └── github-actions.yml
└── docs/
    ├── API.md
    ├── prompt_templates.md
    ├── design_decisions.md
    └── privacy.md
```

API 文档

- 主接口：`POST /api/v1/agent/process` - Agent处理接口（窄协议）
- 查询接口：`POST /api/v1/agent/query` - 查询记忆接口
- 协议文档：详见 `docs/narrow_protocol.md`

部署

支持 Docker 和 Kubernetes 部署，详见 `deploy/` 目录。

开发

```
# 安装依赖
pip install -r requirements.txt

# 运行测试
python test_project.py

# 启动开发服务器
python run_dev.py
```

监控

- Prometheus 指标收集
- Grafana 仪表板
- 结构化日志输出

隐私合规

- 用户授权机制
- 数据最小化原则
- 自动数据保留策略
- 访问控制与审计日志

API设计文档

概述

根据会议讨论，采用窄协议设计，简化Agent和APP之间的通信协议，主要包含：

- `image_list`：base64编码的图像列表（APP端已预处理）
- `task_name`：任务名称，用于业务逻辑判断
- `related_params`：相关参数（地点、情绪、时间戳、车机信号等）

协议设计

1. 请求格式

代码块

```
{
  "image_list": [
    "...",
    "..."
```

```
  ],
  "task_name": "travel_record",
  "related_params": {
    "location": "桂林",
    "emotion": "happy",
    "timestamp": "2024-01-15T10:30:00Z",
    "vehicle_signals": {
      "speed": 60,
      "direction": "north",
      "weather": "sunny",
      "time_of_day": "afternoon"
    },
    "instruction": "记录此刻风景",
    "style": "poetic"
  }
}
```

2. 响应格式

代码块

```
{
  "success": true,
  "task_name": "travel_record",
  "result": {
    "title": "桂林山水甲天下",
    "summary": "在桂林的春日午后，山水如画，令人心旷神怡。",
    "tags": ["guilin", "landscape", "spring", "scenic", "peaceful"],
    "confidence": 0.92
  },
  "memory_id": 123,
  "processing_time": 2.45
}
```

支持的任务类型

1. travel_record - 旅行记录

- **描述**: 生成旅行记忆记录

- **输入**: 风景图像

- **输出**: 标题、摘要、标签

2. scenic_analysis - 风景分析

- **描述**: 专业风景分析

- **输入**: 风景图像

- **输出**: 场景类型、地点特征、时间特征

3. interior_analysis - 舱内分析

- **描述**: 车内环境分析

- **输入**: 舱内图像

- **输出**: 环境状态、人员活动、安全状况

4. event_record - 事件记录

- **描述**: 事件记录

- **输入**: 事件相关图像

- **输出**: 事件类型、参与者、时间地点

5. memory_query - 记忆查询

- **描述**: 查询历史记忆

- **输入**: 查询文本

- **输出**: 相关记忆列表

车机信号参数

vehicle_signals 字段说明

字段	类型	说明	示例
speed	number	车速(km/h)	60
direction	string	行驶方向	"north"
weather	string	天气状况	"sunny"
time_of_day	string	一天中的时间	"afternoon"
temperature	number	温度(°C)	25
humidity	number	湿度(%)	65
road_type	string	道路类型	"highway"
traffic_condition	string	交通状况	"smooth"

业务逻辑判断

根据task_name的业务逻辑

1. **travel_record**

- 使用旅行记录提示词模板
- 重点提取景色、情绪、地点信息
- 生成情感化标题和摘要

2. **scenic_analysis**

- 使用专业风景分析提示词
- 识别景观类型和特征
- 分析时间和环境因素

3. **interior_analysis**

- 使用舱内分析提示词
- 检测人员和安全状况
- 评估环境舒适度

4. **event_record**

- 使用事件记录提示词
- 识别事件类型和参与者
- 记录重要细节

5. **memory_query**

- 直接查询memory模块
- 不调用model server
- 返回相关记忆列表

错误处理

错误响应格式

代码块

```
{
  "error": "错误描述",
  "error_code": "ERROR_CODE",
  "request_id": "req_123",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

常见错误码

错误码	说明	HTTP状态码
INVALID_TASK	不支持的任务类型	400
MISSING_PARAMS	缺少必需参数	400
MODEL_ERROR	Model server错误	503
MEMORY_ERROR	Memory存储错误	500
INTERNAL_ERROR	内部服务器错误	500

示例请求

旅行记录请求

代码块

```
curl -X POST http://localhost:8080/api/v1/agent/process \
  -H "Content-Type: application/json" \
  -d '{
    "image_list": ["..."],
    "task_name": "travel_record",
    "related_params": {
      "location": "桂林",
      "emotion": "happy",
      "timestamp": "2024-01-15T10:30:00Z",
      "vehicle_signals": {
        "speed": 60,
        "weather": "sunny"
      },
      "instruction": "记录此刻风景",
      "style": "poetic"
    }
  }'
```

记忆查询请求

代码块

```
curl -X POST http://localhost:8080/api/v1/agent/process \
```

```
  -H "Content-Type: application/json" \
  -d '{
    "image_list": [],
    "task_name": "memory_query",
    "related_params": {
      "query": "桂林",
      "top_k": 5
    }
  }'
```

性能要求

- **响应时间**: 目标 < 3秒

- **图像数量**: 建议不超过10张

- **图像大小**: 建议不超过1MB每张

- **并发处理**: 支持多请求并发

安全考虑

- 所有请求需要验证参数格式
- 敏感信息在日志中脱敏
- 支持请求频率限制
- 图像数据临时存储，处理完成后清理

系统架构文档

架构概览

根据会议讨论，系统采用模块化设计，主要包含三个核心模块：

代码块

```
┌──────────────────┐    ┌──────────────────┐    ┌──────────────────┐
│       APP        │    │     Agent        │    │   Model Server   │
│    (车机端)      │◄──►│   (交互层)       │◄──►│   (VLLM服务)     │
└──────────────────┘    └──────────────────┘    └──────────────────┘
                                  │
                                  ▼
                          ┌──────────────────┐
                          │     Memory       │
                          │   (SQLite)       │
                          └──────────────────┘
```

模块职责

1. APP模块（车机端）

- **职责**: 图像预处理、用户交互、数据采集

- **功能**:

- 视频抽帧和resize（避免Linux端CPU占用过高）
- 图像base64编码
- 车机信号采集（速度、方向、天气等）
- 用户情绪和指令输入

- **输出**: image_list + task_name + related_params

2. Agent模块（交互层）

- **职责**: 业务逻辑管理、prompt拼接、模块协调

- **功能**:

- 根据task_name进行业务逻辑判断
- 管理prompt模板和拼接
- 调用Model Server进行分析
- 将结果存储到Memory模块
- 处理查询请求

- **核心方法**:

- `process_task()`: 处理任务主入口
- `_handle_model_task()`: 处理需要model的任务
- `_handle_memory_query()`: 处理记忆查询

3. Model Server模块（VLLM服务）

- **职责**: AI模型推理、多模态分析

- **功能**:

- 接收图像和prompt
- 进行多模态分析
- 返回结构化结果（title、summary、tags）

- **协议**: 类似OpenAI协议，但更简洁

4. Memory模块（SQLite存储）

- **职责**: 本地内存管理、多维度信息存储

- **功能**:

- 使用SQLite进行本地存储
- 存储多维度旅行记忆
- 支持场景、动植物、活动等维度查询
- 避免使用RAG，减少管理负担

数据流

1. 旅行记录流程

```
代码块

1   APP → Agent → Model Server → Memory
2     ↓       ↓          ↓          ↓
3   图像预处理 → 业务逻辑 → AI分析 → 本地存储
```

2. 记忆查询流程

```
代码块

1   APP → Agent → Memory
2     ↓       ↓       ↓
3   查询请求 → 业务逻辑 → 本地查询
```

协议设计

窄协议（Agent ↔ APP）

```
代码块

1   {
2     "image_list": ["base64_image1", "base64_image2"],
3     "task_name": "travel_record",
4     "related_params": {
5       "location": "桂林",
6       "emotion": "happy",
7       "timestamp": "2024-01-15T10:30:00Z",
8       "vehicle_signals": {
9         "speed": 60,
```

```
        "weather": "sunny"
      }
    }
  }
```

Model协议（Agent ↔ Model Server）

代码块

```
{
  "images": ["base64_image1", "base64_image2"],
  "prompt": "系统提示词 + 上下文信息",
  "task_name": "travel_record",
  "metadata": {...}
}
```

业务逻辑链路

1. 旅行记录链路

1. APP发送 `travel_record` 任务
2. Agent根据task_name选择prompt模板
3. 注入location、emotion、vehicle_signals等上下文
4. 调用Model Server进行分析
5. 将结果存储到Memory模块
6. 返回处理结果和memory_id

2. 风景分析链路

7. APP发送 `scenic_analysis` 任务
8. Agent使用专业风景分析prompt
9. 重点分析景观类型、地点特征、时间特征
10. 调用Model Server进行专业分析
11. 存储分析结果

3. 记忆查询链路

12. APP发送 `memory_query` 任务
13. Agent直接查询Memory模块

14. 不调用Model Server
15. 返回相关记忆列表

技术特点

1. 预处理优化

- **APP端处理**: 抽帧、resize在APP端完成
- **减少传输**: 避免传输原始视频文件
- **降低延迟**: 减少Linux端CPU占用

2. 模块解耦

- **独立模块**: 各模块职责明确，独立开发
- **标准接口**: 模块间通过标准协议通信
- **易于扩展**: 新增任务类型只需扩展prompt模板

3. 本地存储

- **SQLite**: 轻量级本地数据库
- **多维度**: 支持场景、动植物、活动等多维度存储
- **无RAG**: 避免复杂的向量检索，降低管理成本

4. 轻量级框架

- **Flask**: 替代FastAPI，更接近C语言的简洁性
- **Python运行**: 直接用Python运行，避免uvicorn
- **简单部署**: 减少框架复杂性

部署架构

代码块

```
┌──────────────────┐
│   Nginx/负载均衡  │
└──────────────────┘
          │
┌─────────▼────────┐
│   Flask应用      │
│   (Agent服务)    │
└──────────────────┘
          │
┌─────────▼────────┐
```

```
│    SQLite数据库    │
│    (本地存储)      │
└────────────────────┘
```